Software Verification

Master Programme in Computer Science

Mário Pereira mjp.pereira@fct.unl.pt

Nova School of Science and Technology, Portugal

October 19, 2026

Lecture 6

based on previous editions by João Seco, Luís Caires, and Bernardo Toninho

based on lectures by Andrei Paskevich, Jean-Christophe Filliâtre and Claude Marché

Administrativia

I will not be here next Tuesday.

I will be giving Q&A session for the midterm on Thursday.

If you cannot attend Thursday's class, please contact me directly, ASAP.

Verification of Imperative Programs (3/n)

1. Loop invariants – recap
2. Sorting algorithms
3. Extensions to weakest precondition calculus
   - Functions definition and call
   - Termination
   - Exceptions
4. Verification with bounded integers

Loop invariants – recap

Loop Invariant – Example

Loop invariant approximate state assertions before the loop, between iterations, and at the end of the loop.

```
let max_array (a: array int) : int
  requires { 0 < a.length }
  ensures  { forall k. 0 <= k < a.length -> a[k] <= result }
= let ref m = a[0] in
  let ref i = 1 in
  while i < a.length do
    variant   { ??? }
    invariant { ??? }
    invariant { ??? }
    if m < a[i] then
      m <- a[i];
    i <- i + 1
  done;
  m
```

Loop Invariant – Example

Loop invariant approximate state assertions before the loop, between iterations, and at the end of the loop.

```
let max_array (a: array int) : int
  requires { 0 < a.length }
  ensures  { forall k. 0 <= k < a.length -> a[k] <= result }
= let ref m = a[0] in
  let ref i = 1 in
  while i < a.length do
    variant   { a.length - i }
    invariant { ??? }
    invariant { ??? }
    if m < a[i] then
      m <- a[i];
    i <- i + 1
  done;
  m
```

Loop Invariant – Example

Loop invariant approximate state assertions before the loop, between iterations, and at the end of the loop.

```
let max_array (a: array int) : int
  requires { 0 < a.length }
  ensures  { forall k. 0 <= k < a.length -> a[k] <= result }
= let ref m = a[0] in
  let ref i = 1 in
  while i < a.length do
    variant   { a.length - i }
    invariant { 1 <= i <= a.length }
    invariant { ??? }
    if m < a[i] then
      m <- a[i];
    i <- i + 1
  done;
  m
```

Loop Invariant – Example

Loop invariant approximate state assertions before the loop, between iterations, and at the end of the loop.

```
let max_array (a: array int) : int
  requires { 0 < a.length }
  ensures  { forall k. 0 <= k < a.length -> a[k] <= result }
= let ref m = a[0] in
  let ref i = 1 in
  while i < a.length do
    variant   { a.length - i }
    invariant { 1 <= i <= a.length }
    invariant { forall k. 0 <= k < i -> a[k] < m }
    if m < a[i] then
      m <- a[i];
    i <- i + 1
  done;
  m
```

Binary Search

```
predicate sorted (a: array int)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int) (value: int) : (pos: int)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { ??? }
    invariant { ??? }
    invariant { ??? }
    let ref mid = div (high + low) 2 in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Binary Search

```
predicate sorted (a: array int)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int) (value: int) : (pos: int)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { ??? }
    invariant { ??? }
    let ref mid = div (high + low) 2 in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Binary Search

```
predicate sorted (a: array int)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int) (value: int) : (pos: int)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { forall k. 0 <= k < low -> a[k] <> value }
    invariant { forall k. high <= k < a.length -> a[k] <> value }
    let ref mid = div (high + low) 2 in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Binary Search

```
predicate sorted (a: array int)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int) (value: int) : (pos: int)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { forall k. 0 <= k < a.length -> a[k] = value -> low <= k < high }
                (* low >= high -> a[k] <> value *)
    let ref mid = div (high + low) 2 in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Sorting algorithms

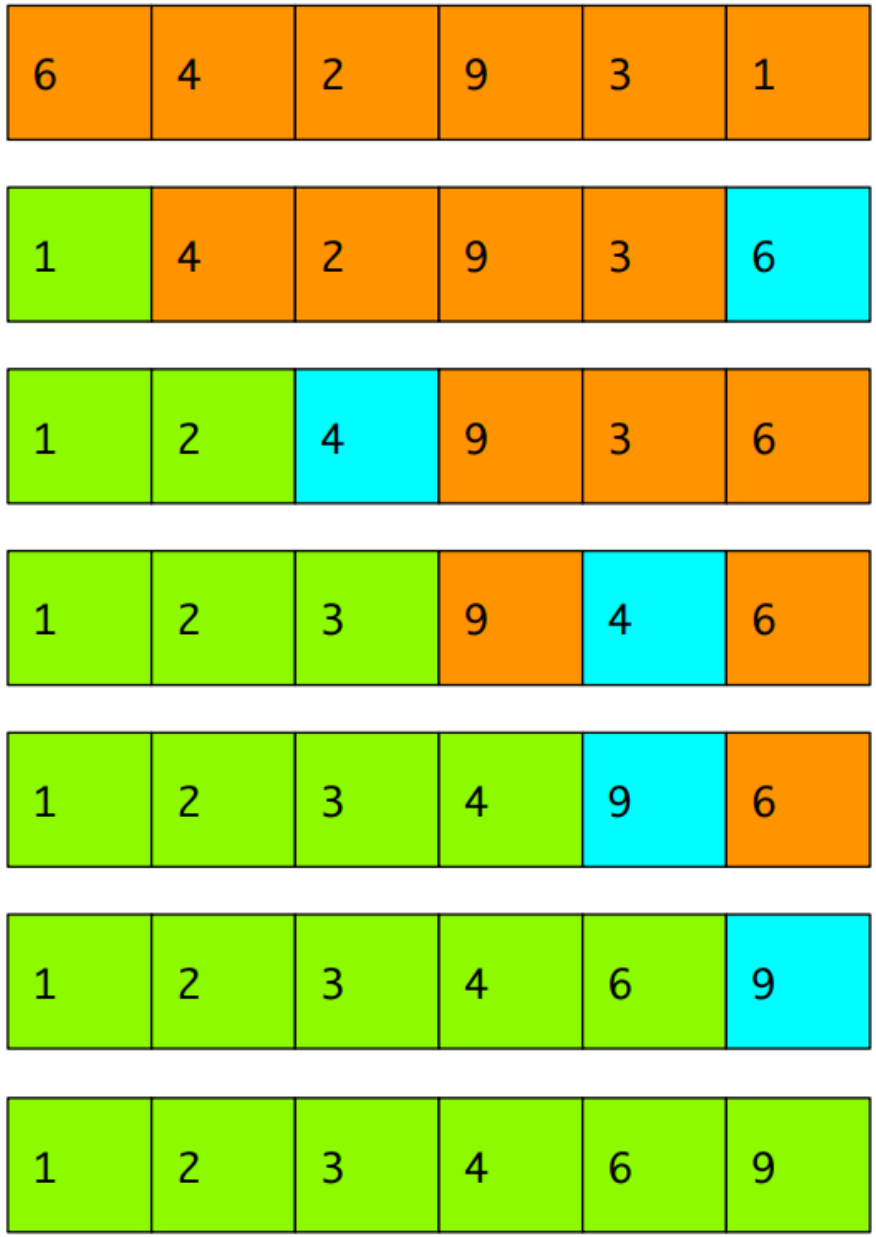
Sorting – rationale

Sorting algorithms often iteratively traverse the data structure gradually sorting its elements.

Loop invariants capture the transient status of the sorting algorithm.

We illustrate that using Selection sort.

Selection sort



Selection sort – implementation

```
let selection_sort (a: array int)

  let min = ref 0 in
  for i = 0 to length a - 1 do



    min := i;
    for j = i + 1 to length a - 1 do


      if a[j] < a[!min] then min := j
    done;
    label L in
    if !min <> i then swap a !min i;
  done
```

Selection sort – specification

```
let selection_sort (a: array int)
  ensures { sorted a
  let min = ref 0 in
  for i = 0 to length a - 1 do
    (* a[0..i[ is sorted; now find minimum of a[i..n[ *)
    invariant { sorted_sub a 0 i /\
        forall k1 k2: int. 0 <= k1 < i <= k2 < length a -> a[k1] <= a[k2] }
    min := i;
    for j = i + 1 to length a - 1 do
      invariant {
        i <= !min < j && forall k: int. i <= k < j -> a[!min] <= a[k] }
      if a[j] < a[!min] then min := j
    done;
    label L in
    if !min <> i then swap a !min i;
  done
```

A digression on run-time errors

Some operations can fail if their safety preconditions are not met:

- arithmetic operations: division par zero, overflows, etc.
- memory access: `NULL` pointers, buffer overruns, etc.
- assertions

A correct program must not fail:

$\{P\}\, e\, \{Q\}$ if we execute e in a state that satisfies P,
then there will be no run-time errors
and the computation either diverges
or terminates normally in a state that satisfies Q

Assertions

A new kind of expression:

$$\begin{array}{rcll} e & ::= & \ldots & \\ & | & \texttt{assert}\ R & \text{fail if } R \text{ does not hold} \end{array}$$

The corresponding weakest precondition rule:

$$\mathrm{WP}(\texttt{assert}\ R, Q) \quad \equiv \quad R \wedge Q \quad \equiv \quad R \wedge (R \to Q)$$

The second version is useful in practical deductive verification.

Selection sort – not quite done yet!

What if `selectionSort` just fills the array with zeros?

The specification we wrote is still provable!

The need for a better specification:

- the elements of the sorted array
- are a permutation of the
- elements in the input array

Selection sort – a complete specification

```
let selection_sort (a: array int)
  ensures { sorted a /\ permut_all (old a) a }
  let min = ref 0 in
  for i = 0 to length a - 1 do
    (* a[0..i[ is sorted; now find minimum of a[i..n[ *)
    invariant { sorted_sub a 0 i /\ permut_all (old a) a /\
        forall k1 k2: int. 0 <= k1 < i <= k2 < length a -> a[k1] <= a[k2] }
    min := i;
    for j = i + 1 to length a - 1 do
      invariant {
        i <= !min < j && forall k: int. i <= k < j -> a[!min] <= a[k] }
      if a[j] < a[!min] then min := j
    done;
    label L in
    if !min <> i then swap a !min i;
  done
```

Selection sort – permutation of elements

Notation **old a**:

- the elements of the array at function call point
- the pre-state

Extensions to weakest precondition calculus

Functions – recap

we may want to delegate some functionality to a function:

$\texttt{let}\ f\ (v_1 : \tau_1) \ldots (v_n : \tau_n) : \tau\ C = e$	defined function
$\texttt{val}\ f\ (v_1 : \tau_1) \ldots (v_n : \tau_n) : \tau\ C$	abstract function

function behaviour is specified with a contract:

$C ::=$	$\texttt{requires}\ P$	precondition
	$\texttt{writes}\ x_1 \ldots x_k$	modified global references
	$\texttt{ensures}\ Q$	postcondition

Example

```
val ref r : int  (* declare a global reference *)

let incr_r (v: int) : int
  requires { v > 0 }
  writes   { r }
  ensures  { result = old r /\ r = old r + v }
= let u = r in
  r <- u + v;
  u
```



postcondition may refer to the prestate

Verifying a function

verification condition

$$\mathbf{VC}(\texttt{let}\ f\ \ldots) \quad \equiv \quad \forall \vec{x}\vec{v}.\, P \rightarrow \mathbf{WP}(e, Q)[\vec{x}^{\circ} \mapsto \vec{x}]$$

where $\vec{x}$ are all global references mentioned in f
$\vec{x}^{\circ}$ are the `old` versions of $\vec{x}$

$$\begin{aligned}\mathrm{VC}(\texttt{incr_r}) \quad \equiv \quad & \forall rv.\, v > 0 \rightarrow \\ & \mathrm{WP}(\texttt{let } u = r \texttt{ in } r := u + v \texttt{ ; } u, \\ & \qquad \texttt{result} = r^\circ \wedge r = r^\circ + v)[r^\circ \mapsto r]\end{aligned}$$

$$\mathbf{VC}(\texttt{incr_r}) \quad \equiv \quad \forall rv.\, v > 0 \rightarrow (r = r^{\circ} \wedge r + v = r^{\circ} + v)[r^{\circ} \mapsto r]$$

$$\mathbf{VC}(\texttt{incr_r}) \quad \equiv \quad \forall rv.\, v > 0 \rightarrow \\ r = r \wedge r + v = r + v$$

Calling a function

One more expression,

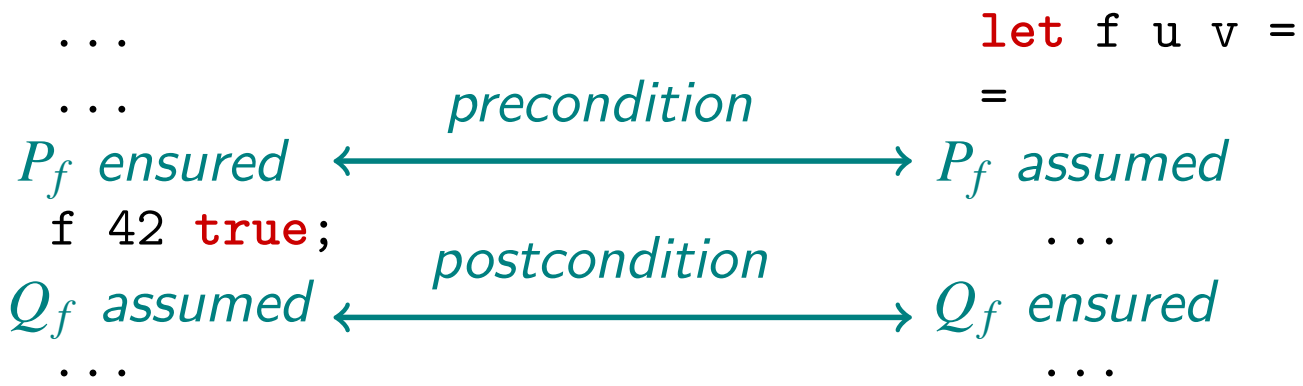
$$\begin{array}{rcll} e & ::= & \ldots & \\ & | & f\ t\ \ldots\ t & \text{function call} \end{array}$$

and its weakest precondition rule:

$$\begin{aligned} \mathrm{WP}(f\ t_1\ \ldots\ t_n, Q) \ \equiv\ & P_f[\vec{v} \mapsto \vec{t}\,] \wedge \\ & (\forall \vec{x}\, \forall \texttt{result}. \\ & \quad Q_f[\vec{v} \mapsto \vec{t}, \vec{x}^{\circ} \mapsto \vec{w}] \to Q)[\vec{w} \mapsto \vec{x}] \end{aligned}$$

P_f	precondition of f	$\vec{x}$	references used in f
Q_f	postcondition of f	$\vec{x}$	references modified in f $(\vec{x} \subseteq \vec{x})$
$\vec{v}$	formal parameters of f	$\vec{w}$	fresh variables

The verification proceeds per function, in a modular way.



This is similar to what a compiler does (e.g. with static typing).

Unsafe operations, again

program function



```
val (/) (x: int) (y: int) : int
  requires { y <> 0 }
  ensures  { result = div x y }
```



logic function

program function



```
val ([]) (a: array 'a) (i: int) : 'a
  requires { 0 <= i < size a }
  ensures  { result = a[i] }
```



logic function

Termination

Termination

Problem: prove that the program terminates for every initial state that satisfies the precondition

It suffices to show that

- every loop makes a finite number of iterations
- recursive function calls cannot go on indefinitely

Terminology: partial correctness + termination = total correctness

Solution: prove that every loop iteration and every recursive call decreases a certain value, called variant, with respect to some well-founded order

For example, for signed integers, a practical well-founded order is

$$i \prec j \quad \stackrel{\text{def}}{=} \quad i < j \wedge 0 \leqslant j$$

Loop termination

A new annotation:

$$
\begin{array}{rcl}
e & ::= & \ldots \\
 & | & \texttt{while}\ t\ \texttt{invariant}\ J\ \texttt{variant}\ t \cdot \prec\ \texttt{do}\ e\ \texttt{done}
\end{array}
$$

The corresponding weakest precondition rule:

$$
\begin{array}{l}
\mathrm{WP}(\texttt{while}\ t\ \texttt{invariant}\ J\ \texttt{variant}\ s \cdot \prec\ \texttt{do}\ e\ \texttt{done}, Q) \equiv \\
\quad J \wedge \\
\quad \forall x_1 \ldots x_k. \\
\qquad (J \wedge\ \ t \rightarrow \mathrm{WP}(e, J \wedge s \prec w)[w \mapsto s]) \wedge \\
\qquad (J \wedge \neg t \rightarrow Q)
\end{array}
$$

$x_1 \ldots x_k$ references modified in e

w a fresh variable (the variant at the start of the iteration)

Termination of recursive functions

A new contract clause:

$$\begin{aligned}\texttt{let rec}\ f\ (v_1 : \tau_1) \ldots (v_n : \tau_n)\ &: \tau \\ &\texttt{requires}\ P_f \\ &\texttt{variant}\ s \cdot \prec \\ &\texttt{writes}\ \vec{x} \\ &\texttt{ensures}\ Q_f \\ =\ &e\end{aligned}$$

For each recursive call of f in e:

$$\begin{aligned}\mathrm{WP}(f\ t_1 \ldots t_n, Q)\ \equiv\ & P_f[\vec{v} \mapsto \vec{t}] \wedge s[\vec{v} \mapsto \vec{t}] \prec s[\vec{x} \mapsto \vec{x}^\circ] \wedge \\ & (\forall \vec{x}\, \forall \texttt{result}. \\ & \quad Q_f[\vec{v} \mapsto \vec{t}, \vec{x}^\circ \mapsto \vec{w}] \to Q)[\vec{w} \mapsto \vec{x}]\end{aligned}$$

$s[\vec{v} \mapsto \vec{t}]$	variant at the call site	$\vec{x}$	references used in f
$s[\vec{x} \mapsto \vec{x}^\circ]$	variant at the start of f	$\vec{w}$	fresh variables

McCarthy 91 function

$$M(n) = \begin{cases} n - 10 & \text{if } n > 100 \\ M(M(n+11)) & \text{if } n \leq 100 \end{cases}$$

Interesting facts:

- $M(n) = 91$, forall $n \leq 100$
- $M(n) = n - 10$, forall $n > 100$.

(Homework: prove this by induction on n.)

A recursive implementation in Why3:

```
let rec mcCarthy (n: int) : int
  ensures { if n <= 100 then result = 91
            else result = n - 10 }
  variant { 101 - n }
= if n > 100 then n - 10
  else mcCarthy(mcCarthy(n + 11))
```

Exceptions

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors
- normal termination: the computation produces a result
  - partial correctness ensures conformance to the contract

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors
- normal termination: the computation produces a result
  - partial correctness ensures conformance to the contract
- exceptional termination: produces *a different kind of result*

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors
- normal termination: the computation produces a result
  - partial correctness ensures conformance to the contract
- exceptional termination: produces *a different kind of result*
  - the contract should also cover exceptional termination

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors
- normal termination: the computation produces a result
  - partial correctness ensures conformance to the contract
- exceptional termination: produces *a different kind of result*
  - the contract should also cover exceptional termination
  - each potential exception E gets its own postcondition Q_{E}

Exceptions as destinations

Execution of a program can lead to

- divergence: the computation never ends
  - total correctness ensures against non-termination
- abnormal termination: the computation fails
  - partial correctness ensures against run-time errors
- normal termination: the computation produces a result
  - partial correctness ensures conformance to the contract
- exceptional termination: produces *a different kind of result*
  - the contract should also cover exceptional termination
  - each potential exception E gets its own postcondition Q_{E}
  - partial correctness: *if* E *is raised, then* Q_{E} *holds*

Example

```
exception Not_found

val binary_search (a: array int) (v: int) : int
  requires { forall i j. 0 <= i <= j < length a -> a[i] <= a[j] }
  ensures  { 0 <= result < length a  /\  a[result] = v }
  raises   { Not_found -> forall i. 0 <= i < length a -> a[i] <> v }
```

Just another semicolon

Our language keeps growing:

$$
\begin{array}{rcll}
e & ::= & \ldots & \\
 & | & \texttt{raise}\ \mathsf{E} & \text{raise an exception} \\
 & | & \texttt{try}\ e\ \texttt{with}\ \mathsf{E} \to e & \text{catch an exception}
\end{array}
$$

WP handles two postconditions now:

$$\mathbf{WP}(\texttt{skip}, Q, Q_{\mathsf{E}}) \equiv Q$$

Just another semicolon

Our language keeps growing:

$$
\begin{array}{rcll}
e & ::= & \ldots & \\
 & | & \texttt{raise}\ \mathsf{E} & \text{raise an exception} \\
 & | & \texttt{try}\ e\ \texttt{with}\ \mathsf{E} \to e & \text{catch an exception}
\end{array}
$$

WP handles two postconditions now:

$$\mathrm{WP}(\texttt{skip}, Q, Q_{\mathsf{E}}) \equiv Q$$

$$\mathrm{WP}(\texttt{raise}\ \mathsf{E}, Q, Q_{\mathsf{E}}) \equiv Q_{\mathsf{E}}$$

Just another semicolon

Our language keeps growing:

$$
\begin{array}{rcll}
e & ::= & \ldots & \\
 & | & \texttt{raise}\ \mathsf{E} & \text{raise an exception} \\
 & | & \texttt{try}\ e\ \texttt{with}\ \mathsf{E} \to e & \text{catch an exception}
\end{array}
$$

WP handles two postconditions now:

$$
\begin{aligned}
\mathrm{WP}(\texttt{skip}, Q, Q_{\mathsf{E}}) &\equiv Q \\
\mathrm{WP}(\texttt{raise}\ \mathsf{E}, Q, Q_{\mathsf{E}}) &\equiv Q_{\mathsf{E}} \\
\mathrm{WP}(e_1\ ;\ e_2, Q, Q_{\mathsf{E}}) &\equiv \mathrm{WP}(e_1, \mathrm{WP}(e_2, Q, Q_{\mathsf{E}}), Q_{\mathsf{E}})
\end{aligned}
$$

Just another semicolon

Our language keeps growing:

$$
\begin{array}{rcll}
e & ::= & \ldots & \\
 & | & \texttt{raise}\ \mathsf{E} & \text{raise an exception} \\
 & | & \texttt{try}\ e\ \texttt{with}\ \mathsf{E} \to e & \text{catch an exception}
\end{array}
$$

WP handles two postconditions now:

$$
\begin{aligned}
\mathrm{WP}(\texttt{skip}, Q, Q_{\mathsf{E}}) &\equiv Q \\
\mathrm{WP}(\texttt{raise}\ \mathsf{E}, Q, Q_{\mathsf{E}}) &\equiv Q_{\mathsf{E}} \\
\mathrm{WP}(e_1\ ;\ e_2, Q, Q_{\mathsf{E}}) &\equiv \mathrm{WP}(e_1, \mathrm{WP}(e_2, Q, Q_{\mathsf{E}}), Q_{\mathsf{E}}) \\
\mathrm{WP}(\texttt{try}\ e_1\ \texttt{with}\ \mathsf{E} \to e_2, Q, Q_{\mathsf{E}}) &\equiv \mathrm{WP}(e_1, Q, \mathrm{WP}(e_2, Q, Q_{\mathsf{E}}))
\end{aligned}
$$

Just another let-in

Exceptions can carry data:

$$
\begin{array}{rcll}
e & ::= & \ldots & \\
 & | & \texttt{raise}\ \mathsf{E}\ t & \text{raise an exception} \\
 & | & \texttt{try}\ e\ \texttt{with}\ \mathsf{E}\ v \to\ e & \text{catch an exception}
\end{array}
$$

Still, all needed mechanisms are already in WP:

$$\mathrm{WP}(t, Q, Q_{\mathsf{E}}) \equiv Q[\texttt{result} \mapsto t]$$

$$\mathrm{WP}(\texttt{raise}\ \mathsf{E}\ t, Q, Q_{\mathsf{E}}) \equiv Q_{\mathsf{E}}[\texttt{result} \mapsto t]$$

$$\mathrm{WP}(\texttt{let}\ v = e_1\ \texttt{in}\ e_2, Q, Q_{\mathsf{E}}) \equiv \mathrm{WP}(e_1, \mathrm{WP}(e_2, Q, Q_{\mathsf{E}})[v \mapsto \texttt{result}], Q_{\mathsf{E}})$$

$$\mathrm{WP}(\texttt{try}\ e_1\ \texttt{with}\ \mathsf{E}\ v \to\ e_2, Q, Q_{\mathsf{E}}) \equiv \mathrm{WP}(e_1, Q, \mathrm{WP}(e_2, Q, Q_{\mathsf{E}})[v \mapsto \texttt{result}])$$

A new contract clause:

$$\begin{aligned}&\texttt{let } f\ (v_1 : \tau_1) \ldots\ (v_n : \tau_n)\ : \tau\\ &\qquad \texttt{requires } P_f\\ &\qquad \texttt{writes } \vec{x}\\ &\qquad \texttt{ensures } Q_f\\ &\qquad \texttt{raises } \mathsf{E} \to Q_{\mathsf{E}f}\\ &= \ e\end{aligned}$$

Verification condition for the function definition:

$$\mathrm{VC}(\texttt{let } f \ldots) \ \equiv \ \forall \vec{x}\vec{v}.\, P_f \to \mathrm{WP}(e, Q_f, Q_{\mathsf{E}f})[\vec{x}^\circ \mapsto \vec{x}]$$

Weakest precondition rule for the function call:

$$\begin{aligned}\mathrm{WP}(f\ \ t_1 \ldots t_n, Q, Q_{\mathsf{E}}) \ \equiv \ & P_f[\vec{v} \mapsto \vec{t}] \ \wedge\\ & (\forall \vec{x}\, \forall \texttt{result}.\, Q_f[\vec{v} \mapsto \vec{t}, \vec{x}^\circ \mapsto \vec{w}] \to Q)[\vec{w} \mapsto \vec{x}] \ \wedge\\ & (\forall \vec{x}\, \forall \texttt{result}.\, Q_{\mathsf{E}f}[\vec{v} \mapsto \vec{t}, \vec{x}^\circ \mapsto \vec{w}] \to Q_{\mathsf{E}})[\vec{w} \mapsto \vec{x}]\end{aligned}$$

Verification with bounded integers

Back to binary search

```
predicate sorted (a: array int)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int) (value: int) : (pos: int)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { forall k. 0 <= k < a.length -> a[k] = value ->
                  low <= k < high }
    let ref mid = div (high + low) 2 in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

A famous bug

In **(high + low) / 2**, what happens if

- **high == MAX_INT**
- **low == MAX_INT - 1**?

INTEGER OVERFLOW!

> *Extra, Extra - Read All About It: Nearly All Binary Searches and Mergesorts are Broken*
>
> Joshua Bloch, Software Engineer (02/06/2006)

So, have we been proving erroneous programs?

Verification and Arithmetic

Until now all our proofs have assumed mathematical integers

- unbounded arithmetic

Our reasoning is sound, provided the code executes using arbitrary precision integers.

- `why3 extract -D ocaml64 bin_search.mlw`
- Use an external library of arbitrary precision integers

What if we want to use 32-bit or 64-bit integers?

What do we know about our implementations?

- Overflow?
- What happens? (Modulo arith., saturated arith., abrupt termination, etc.)

Verification with Machine Integer

Goal: prove that a program is safe with respect to overflows.

32-bit signed integers in two-complement representation: integers between -2^{31} and $2^{31}-1$.

If the mathematical result of an operation fits in the range, that is the computed result.

Otherwise, an overflow occurs:

- Behavior depends on language and environment.

A program is safe wrt overflow if no overflow can occur.

Verification with Machine Integers

Idea: Replace all arith. operations by methods with preconditions.

`x+y` becomes `int32_add(x, y)`, and so on:

```
let int32_add (x y: int) : int
  requires { -2_147_483_648 <= x+y <= 2_147_483_647 }
  ensures  { r = x+y }
= x+y
```

Not great: range constraints of integer must be added explicitly everywhere...

Verification with Machine Integers

Better Idea: Replace int with a range type `int32`:

```
type int32 = < range -0x8000_0000 0x7fff_ffff >

function to_int (x : int32) : int = int32'int x
```

The `to_int` function is a projection back to **int**.

Why3 features other models of precision: `Int63`, `Int64`, `UInt64`, etc.

Can even replace operations by custom methods:

```
predicate in_bounds (n:int) = min <= n <= max

val (+) (a:int32) (b:int32) : int32
  requires { [@expl:integer overflow] in_bounds (a + b) }
  ensures  { result = a + b }
```

Beware! There is an implicit conversion from `int32` to `int`.

Binary Search – Overflow

```
predicate sorted (a: array int32)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int32) (value: int32) : (pos: int32)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0:int32 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { forall k. 0 <= k < a.length -> a[k] = value ->
                  low <= k < high }
    let ref mid = (high + low) / (2:int32) in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Binary Search – No Overflow

```
predicate sorted (a: array int32)
= forall i j. 0 <= i <= j < a.length -> a[i] <= a[j]

let bin_search (a: array int32) (value: int32) : (pos: int32)
  requires { 0 <= a.length /\ sorted a }
  ensures  { 0 <= pos -> pos < a.length /\ a[pos] = value }
  ensures  { pos < 0 -> forall i. 0 <= i < a.length -> a[i] <> value }
= let ref low = 0:int32 in
  let ref high = a.length in
  while low < high do
    variant   { high - low }
    invariant { 0 <= low <= high <= a.length }
    invariant { forall k. 0 <= k < a.length -> a[k] = value ->
                  low <= k < high }
    let ref mid = low + (high - low) / (2:int32) in
    if a[mid] < value then low <- mid + 1
    else if value < a[mid] then high <- mid
    else (* value = a[mid] *) return mid
  done;
  -1
```

Verification with Machine Integers

Caveat: Reasoning with machine integers in this way is challenging.

What if we want to reason with overflow?

Do we really need to always prove absence of overflow?

```
let incr (x: int32) : int32
= x + 1
```

If some variable c

- is initially set to 0
- is only incremented using method `incr`
- it would take 584 years to reach overflow.

Verification with Machine Integers

Caveat: Reasoning with machine integers in this way is challenging.

What if we want to reason with overflow?

Do we really need to always prove absence of overflow?

```
let incr (x: int32) : int32
= x + 1
```

A solution: "*How to avoid proving the absence of integer overflows*".
M. Clochard, J-C. Filliâtre, and A. Paskevich (VSTTE 2015).

General idea:

- only require the absence of overflow during the first n steps
- n being large enough for all practical purposes.